딥러닝 Deep Learning 입문

인공 신경망 ANN, 심층 신경망 DNN

딥러닝 Deep Learning

딥러닝 (심층학습)

인간의 뇌 신경망 구조를 모방한 인공신경망(Artificial Neural Network)을 기반으로, 컴퓨터가 다층 구조를 통해 스스로 데이터의 특징과 패턴을 학습하는 머신러닝의 하위 분야



주요 딥 러닝 모델 유형

ANN (인공 신경망): 인간의 뇌 신경망을 모방한 구조로, 데이터의 패턴을 학습하여 예측, 분류, 패턴 인식 등에 활용

DNN (심층 신경망): 여러 은닉층을 통해 복잡한 패턴을 학습하며 정교한 예측, 분류, 이상 탐지 등에 활용

CNN (합성곱 신경망): 이미지 분류, 얼굴 인식, 자율주행 등 시각 데이터(이미지·영상) 처리 특화

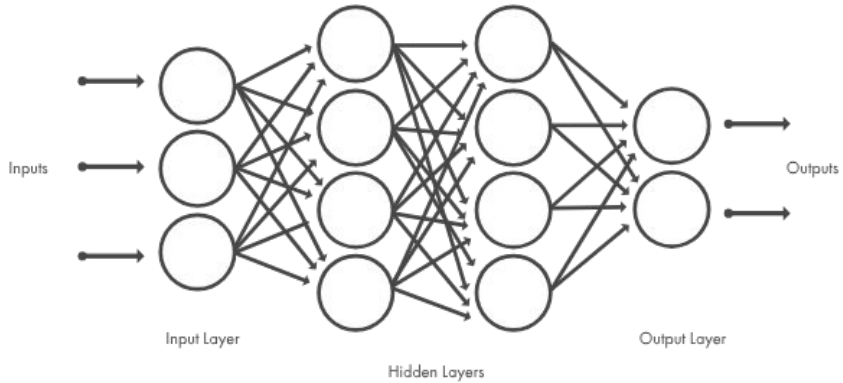
RNN / LSTM (순환 신경망): 음성 인식, 기계 번역, 시계열 데이터 등 시간 순서나 연속성이 있는 데이터 처리 특화

트랜스포머 (Transformer): GPT와 같은 대규모 언어 모델(LLM)과 자연어 처리(NLP)의 핵심이 되는 고성능 구조

딥러닝을 위한 인공 신경망 입문

인공신경망의 발전 단계와 용어

1. 퍼셉트론 Perceptron
2. 다층 퍼셉트론 MLP Multi Layer Perceptron
3. 인공 신경망 ANN Artificial Neural Network
4. 심층 신경망 DNN Deep Neural Network

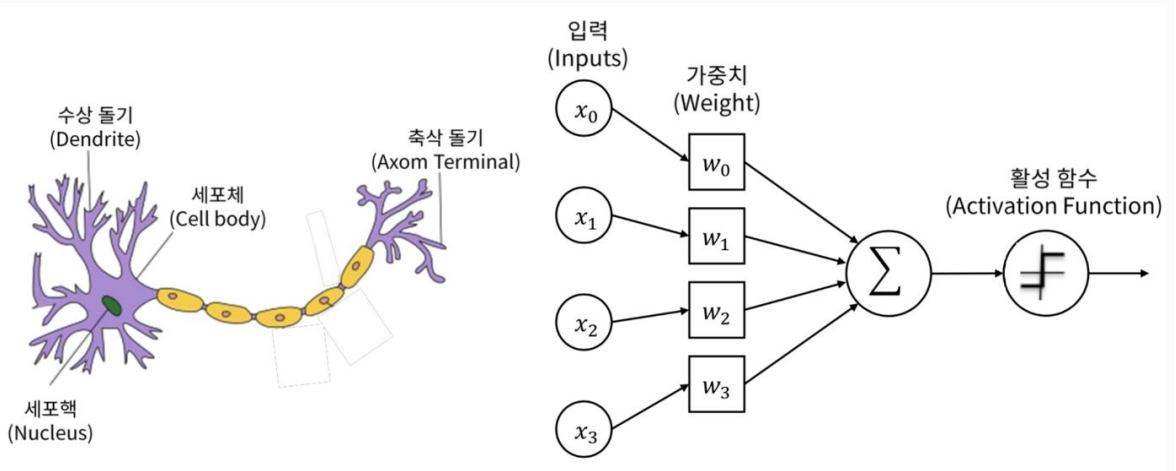


퍼셉트론 Perceptron

퍼셉트론(Percpetron)은 **가장 기본적인 형태의 인공신경망 모델**로, 인간의 신경세포(neuron)의 동작 방식을 단순화하여 수학적으로 모델링한 것이다.

1957년 **Frank Rosenblatt**에 의해 제안되었으며, 입력 데이터를 기반으로 **이진 분류(binary classification)**을 수행하는 **선형 분류기(linear classifier)**로 사용된다.

퍼셉트론은 여러 개의 입력값을 받아 가중치를 곱한 뒤 모두 더하고, 활성화 함수를 통해 최종 출력값을 결정한다.
이 과정은 생물학적 뉴런에서 신호가 전달되는 방식과 유사하다.



- **입력값(Input)** : $x_1, x_2, ..., x_n$
- **가중치(Weight)** : $w_1, w_2, ..., w_n$
- **편향(Bias)** : b
- **활성화 함수(Activiation Function)**

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

$$\hat{y} = f(\mathbf{w} \cdot \mathbf{x} + b), \qquad f(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

퍼셉트론에서는 보통 **계단 함수(step function)**를 사용하여 결과를 0 또는 1로 출력

학습 알고리즘

퍼셉트론은 **퍼셉트론 학습 규칙(Perceptron Learning Rule)**을 이용하여 가중치를 업데이트한다.

학습 과정

1. 입력 데이터를 이용하여 출력값을 계산한다.
2. 예측값과 실제값을 비교한다.
3. 예측이 틀린 경우 가중치를 수정한다.

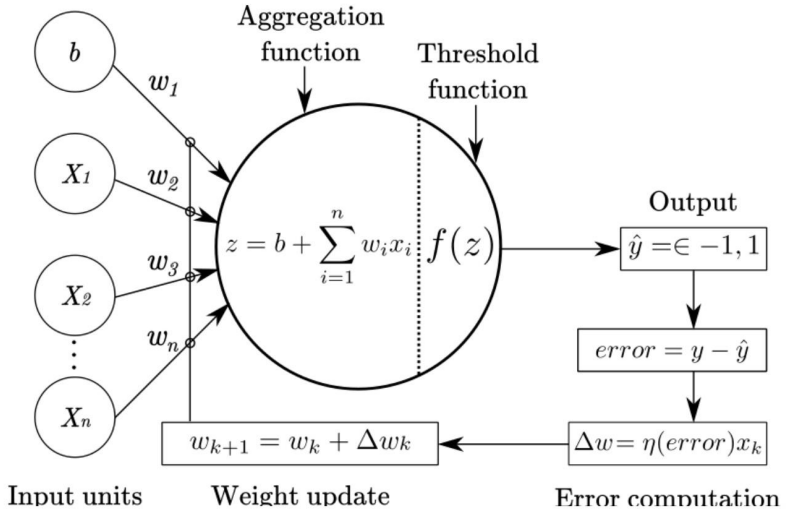
가중치 업데이트

$$w_i = w_i + \eta(y - \hat{y})x_i$$

여기서

- η : 학습률(learning rate)
- y : 실제값
- $\hat{y}$: 예측값

즉, **오분류가 발생했을 때만 가중치를 조정하여 모델을 학습**한다.



Learning rate를 공식으로 표기하거나 수치로 나타낼때는 그리스 문자인 eta를 사용한다.
eta는 η 이런식으로 표기를 하여 사용하는 그리스 문자 H의 소문자이며 오류의 양을 추정하는 데 사용. 열역학적 효율이나 전자기학 등에서 효율(Efficiency)을 나타내는 기호

퍼셉트론의 특징과 한계

퍼셉트론 특징

단층 신경망 구조, 이진 분류 문제 해결 가능, 선형 분류 모델, 계산 구조가 단순함

퍼셉트론의 한계

가장 대표적인 문제는 선형 분리(linearly separable) 문제만 해결할 수 있다는 것이다. 즉, 하나의 직선(또는 초평면)으로 데이터를 분리할 수 있는 경우에만 학습이 가능하다. 예를 들어 **XOR 문제는 선형적으로 분리할 수 없기 때문**에 단일 퍼셉트론으로는 해결할 수 없다.

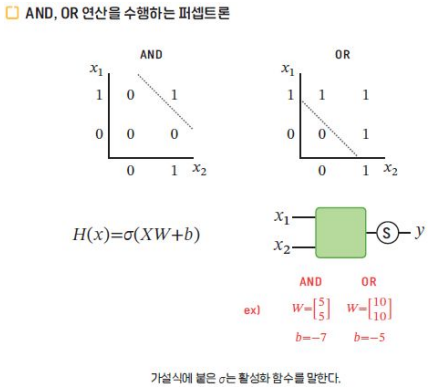
이러한 한계를 해결하기 위해 등장한 모델이 다층 퍼셉트론(MLP)이다.

논리곱

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

논리합

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1



논리 게이트 예

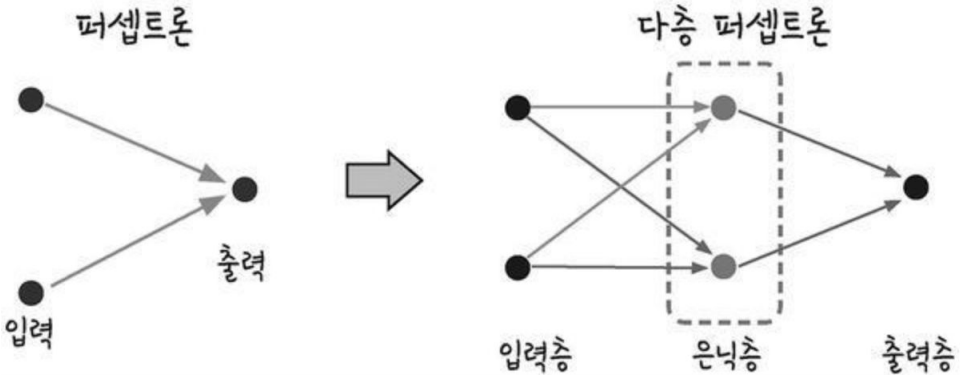
입력이 0과 1뿐인 2입력 퍼셉트론으로 기본 논리 게이트를 만들 수 있다. 아래는 가능한 가중치의 한 예다.

게이트	w_1	w_2	b	결정 경계
AND	1	1	-1.5	$x_1+x_2=1.5$
OR	1	1	-0.5	$x_1+x_2=0.5$
NAND	-1	-1	1.5	$x_1+x_2=1.5$ (부호 반대)
XOR	불가능			직선 하나로 (0,1),(1,0)과 (0,0),(1,1)을 가를 수 없음

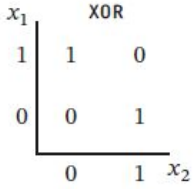
다층 퍼셉트론 MLP Multi Layer Perceptron

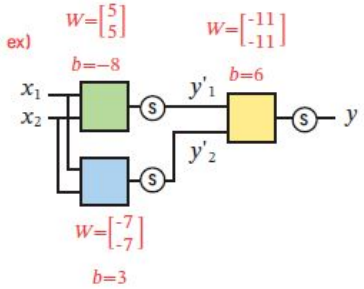
여러 개의 퍼셉트론(인공 뉴런)을 층(Layer) 형태로 쌓아 올린 가장 기본적인 형태의 피드포워드(순전파) 인공신경망. 이런층을 **은닉층**이라고 함.

단일 층만으로는 해결할 수 없었던 비선형 문제를 풀 수 있도록 설계. 심층신경망의 모태가 된 모델 구조.



☐ 다층 퍼셉트론으로 XOR 연산 구현하기





x_1	x_2	y'_1	y'_2	y
0	0	0	1	0
0	1	0	0	1
1	0	0	0	1
1	1	1	0	0

다층 퍼셉트론 구조와 특징

1. 구조 (Architecture)

다층 퍼셉트론은 크게 세 가지 층으로 구성

- **입력층 (Input Layer)**: 외부로부터 원본 데이터나 특성(Feature)을 받아들이는 첫 번째 층
- **은닉층 (Hidden Layer)**: 입력층과 출력층 사이에 존재하는 중간 층으로, 1개 이상 존재할 수 있으며 데이터의 추상적인 특징과 복잡한 비선형 패턴을 학습 (은닉층이 2개 이상인 신경망을 심층 신경망(DNN)이라고 부름)
- **출력층 (Output Layer)**: 은닉층의 연산 결과를 바탕으로 최종 예측값이나 분류 결과를 출력하는 마지막 층

2. 주요 특징 및 핵심 개념

- **비선형성 (Non-linearity)**: 각 뉴런은 입력값의 가중합에 비선형 활성화 함수(계단함수, ReLU, 시그모이드 등)를 적용. 이를 통해 단층 퍼셉트론이 풀지 못했던 XOR 게이트 같은 비선형 분류 문제를 해결할 수 있음
- **순전파 (Feedforward)**: 데이터가 입력층에서 시작해 은닉층을 거쳐 출력층 방향으로 한 방향(루프 없이)으로만 흘러감.
- **역전파 알고리즘 (Backpropagation)**: 출력 결과에서 발생한 오차를 다시 뒤쪽 층으로 거꾸로 전파하면서, 모든 층의 가중치와 편향을 효율적으로 학습·수정

다층 퍼셉트론 학습 방법

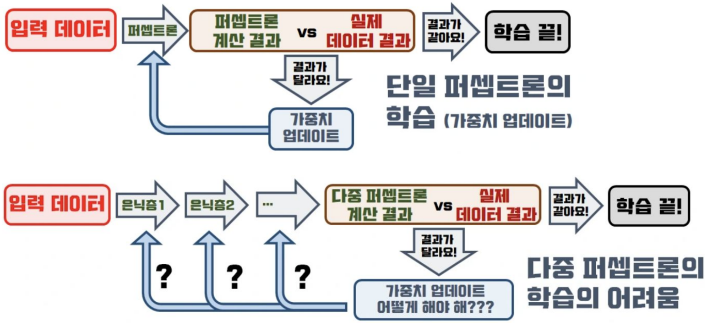
순전파 (Feedforward)와 **역전파 (Backpropagation)**라는 두 가지 핵심 과정을 반복하며 가중치를 최적화하는 방식

1. 순전파 (Forward Propagation)
입력 데이터가 신경망을 통과하며 예측값을 계산한다.

2. 오차 계산 (Loss Calculation)
예측값과 실제값의 차이를 계산한다.

3. 역전파 (Backpropagation)
오차를 출력층에서 입력층 방향으로 전달하며
각 가중치의 기울기를 계산한다.

4. 가중치 업데이트 (Gradient Descent)
계산된 기울기를 이용하여 가중치를 업데이트한다.
이 과정을 반복하면서 모델의 성능이 개선된다.





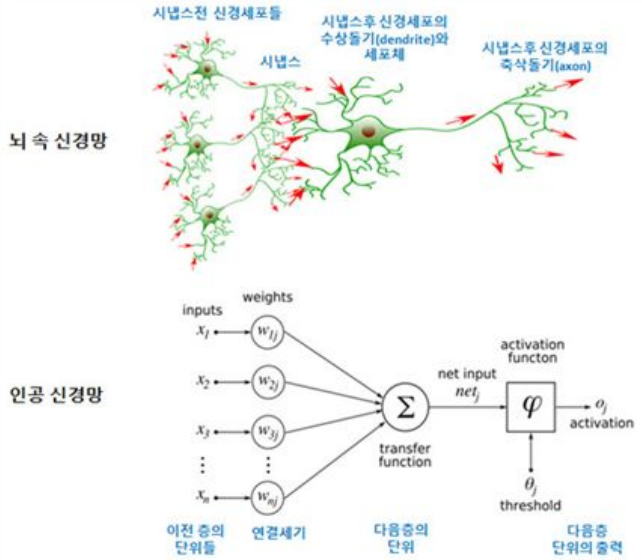
다중 퍼셉트론의 연산 순서와 가중치 업데이트 순서

인공 신경망 ANN

신경망이란, 인간 뇌의 생물학적 특성인 신경 세포, 즉 뉴런neuron의 연결 구조를 말함. 인간의 뇌에는 약 천억 개의 뉴런이 있으며 하나의 뉴런이 다른 뉴런으로부터 신호를 받으면 또 다른 뉴런에 전달하는 방식으로 문제를 인식하고 처리함.

이런 전달 방식을 컴퓨터로 처리할 수 있도록 구현한 것을 **인공 신경망** **artificial neural network** 이라고 부름.

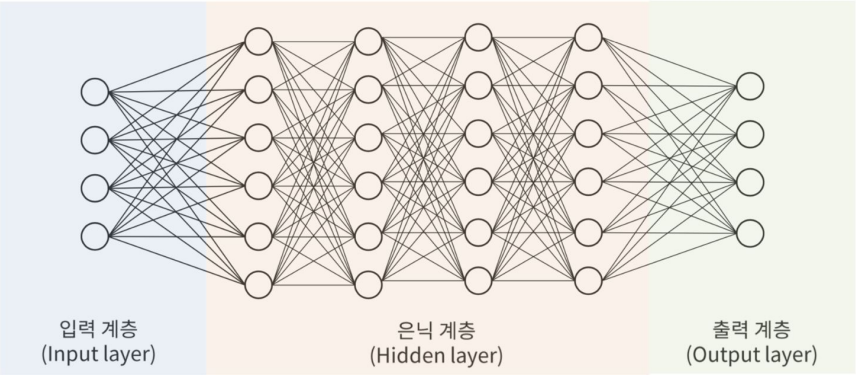
ANN(Artificial Neural Network)은 인간의 신경망 구조를 모방하여 만든 머신러닝 모델의 총칭으로서 특정 하나의 모델을 의미하는 것이 아니라 신경망 기반 모델 전체를 포함

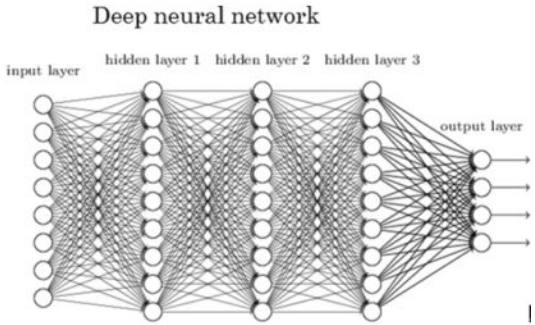


심층 신경망 DNN

인공 신경망에서 여러 입력을 처리해 하나의 결과로 출력하는 가장 고전적인 모델이 **퍼셉트론** perceptron임. 또한 복잡한 문제를 해결하기 위해 은닉층을 추가하는 **다층 퍼셉트론** ML Multi Layer Perceptron이 등장하였으며 이렇게 여러 층이 깊게 쌓여 있다는 의미로 **심층 신경망** DNN, deep neural network이라 부름.

이중 **은닉층이 2개 이상인 심층 신경망을 학습시키는 것이 바로 딥러닝** deep learning 입니다.





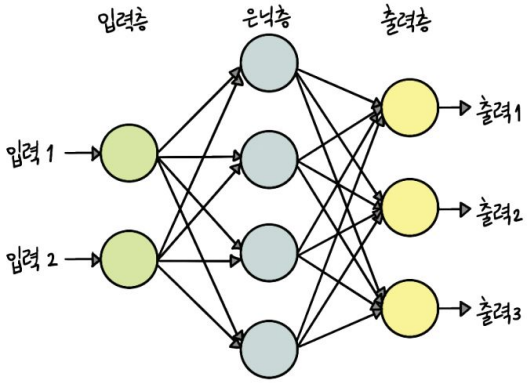
은닉층이 많을수록 더 복잡한 문제를 해결할 수 있습니다.

은닉층과 뉴런을 너무 많이 두면 계산량이 어마어마하게 많아지므로 학습 시간이 오래 걸립니

그리고 세세한 요소에 집중해서 오히려 쉬운 문제를 풀지 못하는 과적합이 발생할 수 있습니다

신경망 Layer(층)별 유닛 개수 의미

은닉층의 레이어, 노드(유닛) 개수를 정하는 공식은 없음. 데이터의 양, 문제의 복잡성, 하드웨어 성능에 따라 실험적으로 구현
통상적으로는 레이어를 층이 깊어질수록 노드 수를 점점 줄여나가며 고차원 특징을 추상화하는 방식이 자주 쓰임.



다층 퍼셉트론의 구조

1. 입력층 (Input Layer) : 입력층의 유닛 개수는 데이터의 피처(Feature, 변수) 개수

집값을 예측할 때 입력 데이터가 '평수, 방 개수, 위치' 3개라면 유닛은 3개.
28x28크기의 흑백 이미지를 1차원 배열로 펼쳐서 입력한다면 유닛은 784개(28*28)

2. 은닉층 (Hidden Layers) : 모델이 데이터로부터 얼마나 다양하고 복잡한 특징(Feature)을 추출할 것인가를 의미하는 개수 (자동 특성 공학)

입력층 유닛이 3개([방 개수, 면적, 연식])이고, 첫 번째 은닉층의 유닛이 4개라고 가정한다면 이 4개의 유닛은 각각 다음과 같은 잠재적 의미(Latent Feature)를 스스로 조합하여 학습 – 예. 공간의 쾌적성, 가족 거주 적합도, 감가상각 및 노후도, 트렌디함/신축 프리미엄 정도
(다음 은닉층이 있다면) – [가족 적합도가 높은데 노후도가 심함 = 리모델링이 필요한 다가구 주택]이라는 더 고차원적이고 복잡한 개념을 완성

3. 출력층 (Output Layer) : 출력층의 유닛 개수는 모델이 최종적으로 해결하려는 문제의 형태(정답의 차원)에 의해 개수 결정

회귀(Regression) 문제: 주가 예측, 집값 예측 등 하나의 연속된 수치를 예측할 때는 보통 1개의 유닛을 사용
이진 분류: 스팸 유무(성공/실패)를 판별할 때는 보통 1개의 유닛(0~1 사이의 확률값 출력)을 사용

특성공학 (파생변수 만들기)

특성공학(Feature Engineering)은 기존 독립변수를 변환하거나 조합하여 모델 학습에 유용한 새로운 특성(파생변수)을 만드는 과정

예) 집값을 예측할 때 다음 세 가지 정보를 사용

- 집 넓이 (area_m2)
- 방 개수 (rooms)
- 건물 연식 (building_age_years)

이 변수들을 조합하면 새로운 정보를 표현할 수 있음.

기존 독립변수 조합	새로운 파생변수	표현하려는 관계
넓이 ÷ 방 개수	방당 평균 면적	공간 활용 특성
넓이 × 방 개수	면적과 방 수의 상호작용	두 특성의 결합 효과
넓이 × 건물 연식	면적과 노후도의 상호작용	연식에 따른 면적 효과의 차이
넓이²	면적의 제곱	넓이와 집값의 비선형 관계

비유: 요리 재료가 같아도 재료를 조합하는 방법에 따라 새로운 맛이 나오듯, 기존 데이터도 조합하면 모델이 활용할 수 있는 새로운 정보가 생길 수 있음.

머신러닝에서의 특성공학 적용

Scikit-learn의 PolynomialFeatures는 기존 변수들을 제곱하거나 서로 곱하여 여러 파생변수를 자동 생성

```
X = df[["area_m2", "rooms", "building_age_years"]]
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
```

3개 독립변수 → 9개 특성

넓이 (A)	방 개수 (R)	연식 (Y)
↓		
PolynomialFeatures(degree=2)		
A	R	Y
A^2	R^2	Y^2
A × R	A × Y	R × Y

원래 특성 3개 + 제곱 특성 3개 + 상호작용 특성 3개 = 총 9개

특성공학의 장점

장점	설명
복잡한 관계 표현	단순 직선으로 표현하기 어려운 관계를 학습할 수 있음
변수 간 상호작용 반영	넓이와 방개수처럼 서로 영향을 주는 특성을 함께 표현
예측 성능 개선 가능	유용한 파생변수를 만들면 MAE, RMSE 등이 개선될 수 있음
기존 모델 활용	별도의 복잡한 모델 없이 선형회귀에도 적용 가능

주의할 점: 특성 수가 늘어난다고 무조건 성능이 좋아지는 것은 아닙니다. 불필요한 파생변수는 과적합을 일으킬 수 있음.

특성공학은 데이터를 무조건 많이 만드는 기술이 아니라, 기존 변수에서 예측에 유용한 관계를 찾아 새로운 특성으로 표현하는 기술

하지만 특성공학은 사람이 새로운 특징을 직접 만들어내는 과정을 수행해야 함

딥러닝의 은닉층을 이용한 자동 특성 추출

자동 특성 추출(Automatic Feature Extraction)

기존의 머신러닝 방법과 달리, 사람이 직접 특성을 추출(Feature Engineering)할 필요 없이, 딥러닝 모델이 데이터로부터 자동으로 유용한 특성을 학습합니다.

구분	특성공학 (Feature Engineering)	딥러닝 은닉층 (Hidden Layer)
새로운 특징 생성	개발자가 직접 만들거나 변환 규칙 지정	신경망이 학습을 통해 생성
예시	넓이², 넓이 × 방 개수	입력값의 가중합과 활성화 함수를 통한 특징
특징의 개수	생성한 파생변수 개수	은닉층의 유닛 개수
특징 생성 방법	정해진 수학적 변환	가중치와 활성화 함수를 학습
핵심	명시적인 특성 변환	자동 특징 학습(Representation Learning)

딥러닝의 은닉층을 이용한 자동 특성 추출



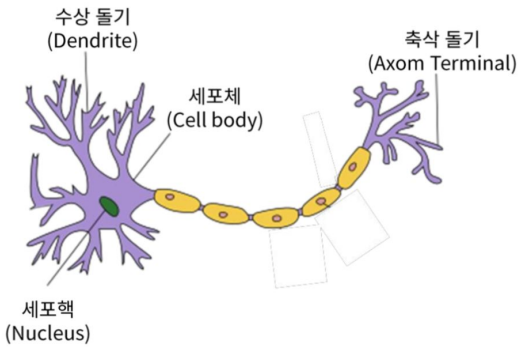
유닛 수가 많아지면 더 다양한 중간 특징을 학습할 수 있는 표현 용량이 커짐.

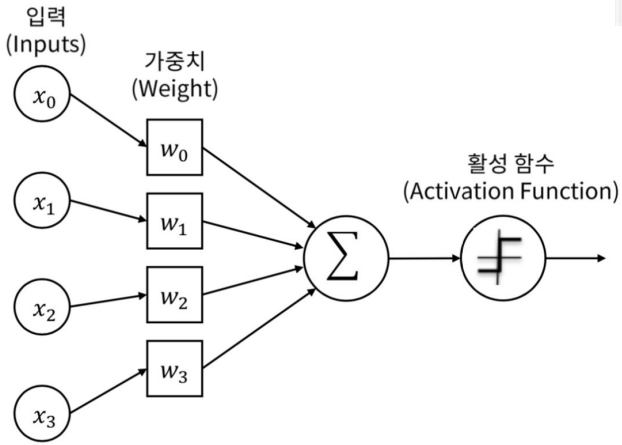
특성공학은 사람이 새로운 특징을 만들어 주는 것이고, 딥러닝 은닉층은 신경망이 유용한 특징을 스스로 학습하도록 하는 것.

유닛 수를 늘려도 성능이 반드시 좋아지지는 않습니다. 모델이 복잡해지면서 과적합 위험이 커질 수 있음
활성화 함수 없이 선형층만 여러 개 쌓으면 전체적으로 하나의 선형 변환과 같음

딥러닝을 위한 인공신경망 개념 정리

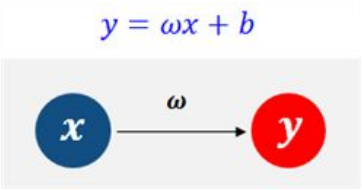
신경 세포



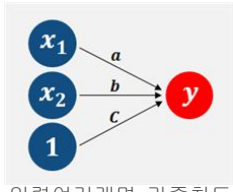


생물학적인 신경 세포를 단순화하여 모델링한 Neuron. 여러 신호를 받아, 하나의 신호를 만들어 전달하는 역할을 한다.

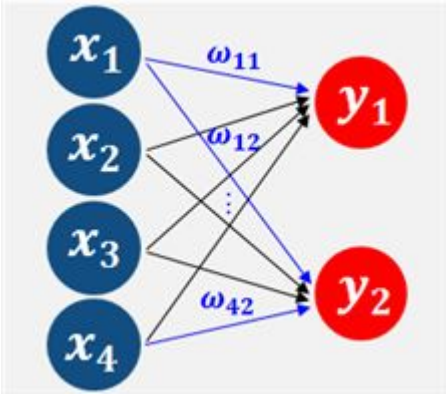
출력을 내기 전에 활성 함수(activation function)을 통해 비선형 특성을 가할 수 있다.



선형회귀 연산



입력여러개면 가중치도 여러개



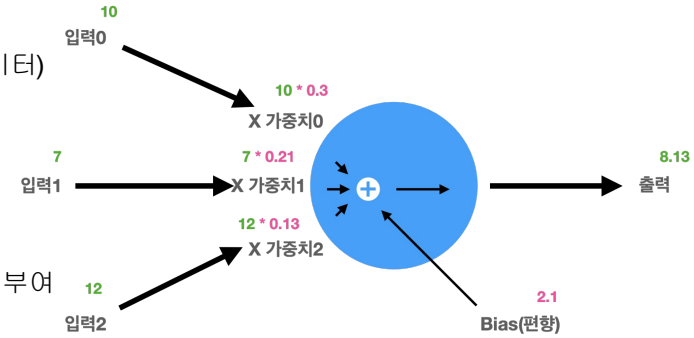
$$\begin{bmatrix} x_1 & x_2 & x_3 & x_4 \end{bmatrix} \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \\ w_{41} & w_{42} \end{bmatrix} = \begin{bmatrix} y_1 & y_2 \end{bmatrix}$$

한층에 유닛이 2개면 그만큼 가중치도 2배

가중치와 편향

가중치(Weight)와 편향(Bias)

입력값을 바탕으로 최종 결과를 예측하기 위해 신경망이 스스로 학습하는 핵심 매개변수(파라미터)



1. 가중치 (Weight, W) : 입력 신호가 얼마나 중요한지를 조절하는 값

입력 데이터에 곱해지며, 1차 함수에서의 기울기와 같은 역할을 함

가중치가 클수록 해당 입력값이 결과에 미치는 영향력이 커짐

예시: 고양이와 개를 분류하는 AI라면, "귀의 모양"이나 "코의 형태" 같은 특징에 더 높은 가중치 부여

2. 편향 (Bias, b) : 뉴런이 얼마나 쉽게 활성화될지를 조절하는 기준점(임계값)입니다.

가중치와 입력값을 곱한 값에 더해지며, 1차 함수에서의 y절편과 같은 역할을 함

은닉층의 뉴런(유닛,노드) 마다 1개씩 존재 $y= w1x1 + w2x2 + \dots + b$

예시: 스팸 메일 필터에서 편향값을 높게 설정하면, 작은 의심만 있어도 스팸으로 분류하도록 기준을 까다롭게 만들 수 있음.

뉴런에서 다른 뉴런으로 전달되는 신호의 세기는 정보의 중요도에 따라 달라질 수 있음.

중요하지 않은 정보는 낮은 가중치로 전달되거나 소멸되며 가중치가 0이 될 수도 있음

편향bias은 마치 라면에서 마지막에 얹는 치즈 같은 역할을 함. 더 좋은 결과, 원하는 결과가 나오도록 약간 조정해 주는 것.

가중치와 편향은 사람이 입력하는 값이 아님. 신경망을 학습한다는 것은 결국 가중치와 편향의 값을 찾아가는 과정.

활성화 함수

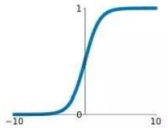
활성화 함수

인공신경망에서 뉴런으로 들어오는 입력값들의 가중치 합을 최종 출력값으로 변환해 다음 층으로 전달하는 비선형 계산식

10점이면 가장 맛있는 라면이고 1점이면 가장 맛없는 라면이라고 할 때, 8점 이상이면 1,
8점 미만이면 0을 결과로 출력하는 활성화 함수를 추가하면 퍼셉트론이 이 함수를 거쳐 맛있는
라면은 1, 맛없는 라면은 0이라는 결과를 출력하게 됨.
이를 은닉층에 사용하면 1인 정보는 다음 층으로 전달되고 0인 정보는 전달되지 않게 됨.
활성화 함수는 망을 더 복잡하고 깊게 만들어 더 복잡한 문제를 잘 풀기 위해서 사용

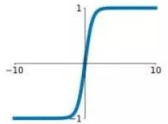
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



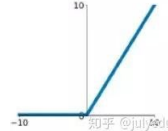
tanh

$\tanh(x)$



ReLU

$\max(0, x)$



뉴런의 출력을 비선형적으로 변환해 모델이 복잡한 패턴을 학습할 수 있게 해주는 핵심 요소
대표적인 활성화함수 : 시그모이드, Softmax, Relu

Sigmoid (시그모이드): 입력값을 0과 1 사이의 S자 형태로 압축. 주로 이진 분류의 출력층에 사용.

Tanh (Hyperbolic Tangent): 입력값을 -1과 1 사이로 압축하며, 중심값이 0이라 시그모이드보다 학습 효율이 조금 더 좋음.

Softmax (소프트맥스): 다중 클래스 분류에서 여러 개체별 확률값의 총합이 1이 되도록 변환합니다.

ReLU (Rectified Linear Unit): 입력값이 0 이하면 0을 출력하고, 0보다 크면 입력값 그대로 출력. 계산이 빨라 현재 가장 많이 사용

딥러닝의 핵심 개념과 구조

계층 구조 (Layers)

딥 러닝 모델은 데이터를 받아들이는 **입력층 (Input Layer)**, 중간에서 복잡한 특징을 스스로 분석하고 학습하는 여러 개의 **은닉층 (Hidden Layers)**, 그리고 최종 결과를 내놓는 **출력층 (Output Layer)**으로 구성

자동 특징 추출 (Feature Learning)

전통적인 머신러닝은 사람이 직접 데이터의 특징(Feature)을 설계해 주어야 했지만, 딥 러닝은 깊게 쌓인 은닉층을 통해 컴퓨터가 원시 데이터에서 직접 유용한 특징을 찾아냄

가중치 조정 (Weights & Backpropagation)

입력값에 가중치를 곱하고 활성화 함수를 거쳐 결과를 예측한 뒤, 정답과의 오차(손실)를 계산하며
이후 역전파(Backpropagation)와 경사 하강법(Gradient Descent) 알고리즘을 이용해 오차가 줄어들도록 수백만~수십억 개의 가중치를 반복해서 수정

머신러닝과 딥러닝의 차이

비교 항목	머신 러닝 (Machine Learning)	딥 러닝 (Deep Learning)
특징 추출	사람이 직접 설계 필요	모델이 데이터로부터 **자동 학습**
데이터 양	상대적으로 적은 데이터로도 가능	**대규모 빅데이터** 필요
필요 연산량	상대적으로 낮음	매우 높음 (**GPU 등 고성능 하드웨어 필수**)
대표 알고리즘	서포트 벡터 머신(SVM), 랜덤 포레스트 등	CNN, RNN, 트랜스포머(Transformer) 등

딥러닝 프레임워크

인공신경망을 쉽게 구축하고 학습할 수 있도록 돕는 소프트웨어 라이브러리

대표적인 딥러닝 프레임워크

TensorFlow: Google에서 개발한 오픈 소스 딥러닝 프레임워크. Keras라는 고수준 API를 제공하여 딥러닝 모델을 쉽게 구축가능

대표 라이브러리 : Keras, TensorFlow Lite, TensorFlow Serving, TensorFlow.js, TensorFlow Extended(TFX - MLOps 플랫폼)
모바일(Android/iOS) 및 웹 배포가 강점

PyTorch:Meta(Facebook)에서 개발한 오픈 소스 딥러닝 프레임워크. 동적 계산 그래프를 사용하여 유연하고 직관적인 모델 개발이 가능

대표 라이브러리 : torchvision, torchaudio, torchtext, PyTorch Lightning(학습고도화), Hugging Face Transformers, Detectron2, Ultralytics YOLO
AI 연구 논문의 예제 대부분이 PyTorch로 제공

딥러닝 프레임워크의 특징

자동 미분: 신경망 학습에 필요한 미분(기울기)을 자동으로 계산

GPU 가속: GPU를 사용하여 딥러닝 모델의 학습 및 추론 속도를 높일 수 있음

다양한 모델 및 레이어 제공: CNN, RNN 등 다양한 딥러닝 모델과 레이어를 제공

모델 저장 및 불러오기 : 학습된 모델을 저장하고, 다시 불러와서 사용 가능

실습순서

1. 파이토치

1. 파이토치 기본
2. 회귀 Regression (ANN, DNN)
3. 분류 Classification (이진분류, 다중분류)

2. Tensorflow

1. 회귀 Regression (선형, 비선형, 다중)
2. 분류 Classification (이진분류, 다중분류)